

Probeklausur: Mikrocomputertechnik 1

Bearbeitungszeit: 90 Minuten · Gesamtpunktzahl: 100 Punkte

Musterlösung

Name: _____ Matrikelnummer: _____

Kurs / Studiengruppe: _____

Erlaubte Hilfsmittel: Das offizielle RISC-V Green Card / Reference Sheet (wird ausgeteilt).

Aufgabe	Thema	Punkte	erreicht
1	Zahlendarstellung	10	
2	Code-Analyse / Tracing	15	
3	C-to-Assembly	25	
4	Funktionen / Stack (Kurzfragen)	15	
5	Steuerwerk / Datenpfad	15	
6	Pipelining / Hazards	20	
	Summe	100	

Aufgabe 1: Zahlendarstellung (10 Punkte)

1. Wandeln Sie 85_{10} in eine 8-Bit-Binärzahl und in Hexadezimal um. (4 Pkt)
2. Bilden Sie das 8-Bit-Zweierkomplement von -85 . (4 Pkt)
3. Welches Byte liegt in Little-Endian an Adresse $0x1000$, wenn das Wort $0xAABBCCDD$ ab $0x1000$ gespeichert wird? (2 Pkt)

Musterlösung Aufgabe 1 (10 Punkte)

1. $0101\ 0101 = 0x55$ (4 Pkt)
2. Invertieren $1010\ 1010$, $+1 \rightarrow 1010\ 1011 = 0xAB$ (4 Pkt)
3. Least Significant Byte DD (2 Pkt)

Aufgabe 2: Code-Analyse / Tracing (15 Punkte)

Alle Register starten bei 0. Welchen dezimalen Wert enthalten $t0$, $t1$ und $t2$ nach Ausführung?

```
li t0, 5
li t1, 3
sub t2, t0, t1
slli t2, t2, 2
add t0, t0, t2
```

- $t0 = \underline{\hspace{2cm}}$ (8 Pkt)
- $t1 = \underline{\hspace{2cm}}$ (2 Pkt)
- $t2 = \underline{\hspace{2cm}}$ (5 Pkt)

Musterlösung Aufgabe 2 (15 Punkte)

- $t0 = 13, t1 = 3, t2 = 8$
- Trace: $5 - 3 = 2, 2 \ll 2 = 8, 5 + 8 = 13$

Aufgabe 3: C-to-Assembly (25 Punkte)

Übersetzen Sie den C-Code in RISC-V-Assembler. Nutzen Sie `s0` für `sum`, `t0` für `i`. Invertierte Abbruchbedingung (`bge` / `bgt`).

```
int sum = 0;
int i = 1;

while (i <= 10) {
    sum = sum + i;
    i++;
}
```

Ihre Assembler-Lösung

Musterlösung Aufgabe 3 (25 Punkte)

```
li s0, 0          # sum = 0          (4 Pkt)
li t0, 1          # i = 1            (3 Pkt)
li t1, 11         # Limit für bge    (3 Pkt)
While_Start:
    bge t0, t1, While_End  # i >= 11? (6 Pkt)
    add s0, s0, t0        # sum = sum + i (4 Pkt)
    addi t0, t0, 1        # i++        (3 Pkt)
    j While_Start         (2 Pkt)
While_End:
```

Alternative: `li t1, 10` und `bgt t0, t1, While_End`.

Aufgabe 4: Funktionen und Stack (15 Punkte)

Beantworten Sie kurz:

1. Welche Register übergibt der Caller typischerweise für die ersten beiden Argumente? (3 Pkt)
2. Ist `ra` Caller-saved oder Callee-saved — und warum muss eine Non-Leaf-Funktion `ra` sichern? (6 Pkt)
3. Skizzieren Sie den Prolog einer Leaf-Funktion, die nur `s0` auf dem Stack sichert (Wortgranularität wie in Venus; reale ABI: 16-Byte-Alignment). (6 Pkt)

Musterlösung Aufgabe 4 (15 Punkte)

1. `a0, a1` (3 Pkt)
2. `ra` ist **Caller-saved** (bzw. gehört dem Caller); eine Non-Leaf-Funktion überschreibt `ra` beim eigenen `jal` und muss den alten Rückkehrwert daher im Prolog auf dem Stack sichern (6 Pkt).
3. Beispiel (4 Pkt Prolog + 2 Pkt Epilog-Idee):

```
addi sp, sp, -4
sw    s0, 0(sp)
# ... Rumpf ...
lw    s0, 0(sp)
addi sp, sp, 4
ret
```

Aufgabe 5: Steuerwerk / Datenpfad (15 Punkte)

Füllen Sie die Wahrheitstabelle für `sub` und `lw` aus (0, 1 oder *X*). Signalnamen nach Patterson & Hennessy (Kurskonvention).

Instruktion	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
<code>sub</code>						
<code>lw</code>						

Musterlösung Aufgabe 5 (15 Punkte; 1 Pkt pro Zelle + 3 Pkt für vollständige korrekte Zeilen)

Instruktion	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
<code>sub</code>	0	0	1	0	0	0
<code>lw</code>	1	1	1	1	0	0

Aufgabe 6: Pipelining und Hazards (20 Punkte)

5-stufige Pipeline mit Hazard Detection und Forwarding. Füllen Sie das Timing-Raster aus und markieren Sie Stalls.

```
lw t0, 0(a0)
addi t0, t0, 5
add t1, t0, t0
```

Befehl / Takt	1	2	3	4	5	6	7	8
<code>lw t0, 0(a0)</code>								
<code>addi t0, t0, 5</code>								
<code>add t1, t0, t0</code>								

Musterlösung Aufgabe 6 (20 Punkte)

Befehl \ Takt	1	2	3	4	5	6	7	8
<code>lw t0, 0(a0)</code>	IF	ID	EX	MEM	WB			
<code>addi t0, t0, 5</code>		IF	ID	STALL	EX	MEM	WB	

Befehl \ Takt	1	2	3	4	5	6	7	8
add t1, t0, t0			IF	STALL	ID	EX	MEM	WB

- lw-Durchlauf korrekt (4 Pkt)
- Load-Use-Stall in Takt 4 für **addi** (8 Pkt)
- Verschiebung von **add** (4 Pkt)
- Zwischen **addi** und **add** reicht Forwarding (4 Pkt)

Korrekturhinweis: Bubble / wiederholtes ID bzw. IF statt „STALL“ sind äquivalent, wenn die Timing-Lage stimmt.